

05-docker-realtime.md

CORE IDEA (Don't skip this)

Docker is **not** about Ubuntu/CentOS/Nginx.

👉 Docker = “pack + run app with everything it needs”

REAL-WORLD MAP (5th class level)

Think like this:

Real World	Docker
Recipe	Dockerfile
Kitchen steps	RUN
Ingredients	Base image (Ubuntu, nginx, etc.)
Final dish	Docker Image
Serving the dish	Container

STEP-BY-STEP MAP TO YOUR CODE

1 Dockerfile = Recipe

Example:

```
FROM ubuntu
RUN apt update -y
RUN apt install apache2 -y
COPY index.html /var/www/html
CMD ["/usr/sbin/apachectl", "-D", "FOREGROUND"]
```

Map it:

- FROM ubuntu → Take raw kitchen (base OS)
- RUN apt install apache2 → Cook (install software)
- COPY index.html → Add your content
- CMD → How to start when served

2 Build = Cooking

`docker build -t my-apache .`

👉 Turns recipe → dish (IMAGE)

3 Run = Serve

`docker run -d -p 8080:80 my-apache`

👉 Starts container (live app)

! BIG MISTAKE YOU MADE

✗ “this we call as cgroup”

No.

👉 **cgroups ≠ Dockerfile**

- **cgroups** = Linux resource limiter (CPU, memory)
 - Docker **uses** cgroups internally
 - You don't write cgroups here
-

🧱 IMAGE LAYERS (THIS IS IMPORTANT)

You said:

“5 layers”

That's partially right — but your reasoning is weak.

RULE:

- 👉 Every **RUN, COPY, ADD** = new layer
 - 👉 Metadata (**MAINTAINER**, **EXPOSE**) = no layer impact
-

Your example:

FROM ubuntu

RUN apt update -y

RUN touch mustafa.txt

```
RUN apt install apache2 -y
COPY index.html /var/www/html
```

👉 Layers:

1. Base (ubuntu)
2. apt update
3. touch file
4. install apache
5. copy file

✓ Yes = 5 layers



REAL PROBLEM (YOU MISSED THIS)

Your Dockerfiles are **bad for production**.

Why?

- 👉 Too many layers
- 👉 Bigger image size
- 👉 Slower CI/CD



HOW REAL ENGINEERS WRITE THIS

Instead of:

```
RUN apt update -y
RUN apt install apache2 -y
```

👉 Do:

```
RUN apt update && apt install -y apache2 && rm -rf /var/lib/apt/lists/*
```

- ✓ 1 layer instead of 2
- ✓ Smaller image



BEST PRACTICE YOU KIND OF DISCOVERED

You noticed:

nginx base image is smaller

Correct.

Compare mentally:

This (bad)

FROM ubuntu

RUN apt install nginx

This (good)

FROM nginx

Why?

- Already optimized
- Smaller
- Faster build

YOUR CASES SIMPLIFIED

Case 1–3 (Ubuntu, CentOS, Rocky)

Same idea:

- Install httpd
- Copy file
- Run server

 These are **learning examples**, not production

Case 4–5 (nginx)

Better approach:

- Use base nginx
- Just copy static files

 Fast

 Clean

 Industry standard

Case 7–8 (Git projects)

👉 This is real-world pattern:

git clone repo

docker build

docker run

✓ Works because:

- Static site
- No dependencies

⚡ KEY CONCEPT YOU SHOULD LOCK

🧠 STATIC vs DYNAMIC

Type	Example	Docker approach
Static	HTML site	nginx base image
Dynamic	Node.js	Node image + build

🧹 IMAGE CLEANUP

You mentioned:

docker image prune

👉 Deletes:

- dangling images (none:none)

✓ Correct

🧠 FINAL MEMORY TRICK

Think in **3 words only**:

👉 **FROM → RUN → CMD**

Everything else is optional.

🚀 IF YOU WANT TO LEVEL UP (REAL GAP)

You didn't touch:

- Multi-stage builds **!** (huge in real world)
 - Alpine images **!** (smallest size)
 - Caching strategy **!** (huge for CI/CD)
 - Security **!** (you're running as root everywhere)
-

BLUNT TRUTH

Right now:

- You're copying tutorials
 - Not optimizing
 - Not thinking in layers or size
-

WHAT YOU SHOULD DO NEXT

Try this:

 Build SAME app in 3 ways:

1. ubuntu + nginx install
2. nginx base image
3. nginx:alpine

Then compare:

docker images

 That will teach you more than theory.

If you want, I can:

- rewrite your Dockerfiles into **production-grade**
- or show **multi-stage + CI/CD optimized version**